

Two Dimensional

Class 12 : Two Dimensional Arrays 2

Q1. Given a row wise & column wise sorted matrix, find whether element K is present or not.

	0	1	2	3
A = 0	-5x	(-2) ←	(1) ←	(13)
1	-4	(0) ✓	3 x	14 x
2	-3	2	6 x	18 x

~~K = 13~~

K = 0 ✓

~~K = -1~~

① Brute force: → i, j , check $a[i][j]$ TC → $O(N \times M)$

②

13 > 0

→ go to smaller value

decreasing element ← 13

elements are ↑ing so K=0 will not be present here.

we start @ 13, since $0 < 13$, go left.

↓

we ignore 14, 18 values.

-5 -2 (1) 13

↓

Now for '0' → we have to go left.

-5 (2) (1) 13

x x
x x

ignore the left part.

$\begin{matrix} \textcircled{-5} \\ -4 \\ -3 \end{matrix} \quad \begin{matrix} \textcircled{-2} \\ \textcircled{0} \\ 2 \end{matrix}$

here $-2 < 0$

↓

we should go down.

↓

our new matrix will look like

0	1
1	-4 0
2	-3 2

Code

$i=0, j=m-1$ // top right,

while ($i < n$ & $j >= 0$)

if ($a[i][j] == k$) return True

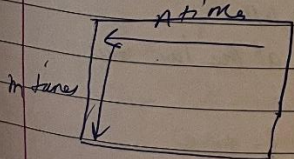
else if ($a[i][j] > k$) $j--$; // go left

else $i++$; // go down

return False

TC: $O(N+M)$

max time will be when you iterate from $i \rightarrow 0$ to $n-1$ & $j \rightarrow m-1$ to 0 → m times.



Q 2 Given the matrix, print boundary elements in clockwise direction

	0	1	2	3	4	5
0	1	2	3	4	5	
1	6	7	8	9	10	
2	11	12	13	14	15	
3	16	17	18	19	20	
4	21	22	23	24	25	

$N-1, 0$

$N-1, N-1$

OR print elements in spiral format.

Op: 1 2 3 4 5 10 15 20 25 24 23 22 21 16 11 6

Code

```

i=0, j=0
for (k=0 to n-2) {
    print(a[i][j])
    j++ // left to right
}
for (k=0 to n-2) {
    print(a[i][j])
    i++ // top to down
}
for (k=0 to n-2) {
    print(a[i][j])
    j-- // right to left at the last row
}
for (k=0 to n-2) {
    print(a[i][j])
    i-- // bottom to top at the 1st column
}

```

TC = $O(N-1 + N-1 + N-1 + N-1)$

$= O(N)$

SC = $O(1)$

Q3

LeetCode moving challenge:
Print element in spiral order in clockwise direction. (square matrix)

M	T	W	T	F	S	S
Page No.:						
Date:						YOUVA

	0	1	2	3	4
0	1	2	3	4	5
1	6	7	8	9	10
2	11	12	13	14	15
3	16	17	18	19	20
4	21	22	23	24	25

O/p: 1 2 3 4 5 10 15 20 25 24 23 22 21 16 11 6
7 8 9 14 19 18 17 12 13

For this we started printing from $r=0, c=0$, our new row & col will be
 $i=1, j=1$, size = 3

	row	column	N
+1	0	0	5
	1	1	3
+1	2	2	1

Code
 $n=0, c=0$

 while ($N > 1$) {

 $i=r, j=c$

 for ($k=0$ to $n-2$) {

 print $a[i][j]$
 $j++$

}

 for ($k=0$ to $n-2$) {

 print $a[i][j]$
 $i++$

}

 for ($k=0$ to $n-2$) {

 print $a[i][j]$
 $j--$

}

 for ($k=0$ to $n-2$) {

 print $a[i][j]$
 $TC: O(N^2)$

or

 $O(N \times M)$ for square

 $SC: O(1)$

i--;

r++ ; c++ , N-- = 2

if (N == 1) print (a[r][c]) } when N == 1, eg. 1 = 1 while loop will break & 13 will be printed separately.

This will come when we have N as odd matrix,

↓

In case of even matrix

1	2	3	4	N=4
5	6	7	8	N=2
9	10	11	12	N=0
13	14	15	16	

SUB MATRIX

Subarray : continuous part of array
 submatrix : " " " a matrix

0	1	2	3	
0	1	2	3	4
1	5	6	7	8
2	9	10	11	12
3	13	14	15	16

submatrix represents

Top left = (1,2)
 bottom right = (3,4)

Q1 Find the # of submatrices in which a[i][j] is present for given i, j.

Example: In how many submatrices, element 6 is present

	0	1	2
0	1	2	3
1	4	5	6
2	7	8	9

$(i, j) = (1, 2)$

1 2 3 2 3 3 4 5 6 5 6 6

4 5 6 5 6 6

4 5 6 5 6 6 1 2 3 2 3 3

7 8 9 8 9 9 4 5 6 5 6 6

7 8 9 8 9 9

ans = 12. (total 12 submatrices containing '6' as element)

$i=2$ $j=3$

1	2	3	4	5
6	7	8	9	10
11	12	13	14	15
16	17	18	19	20
21	22	23	24	25

possible top left cell = $(i+1) \times (j+1)$

for $k=14$
when $i=2, j=3$
 $= (2+1) \times (3+1)$

$i=2$ $j=3$

1	2	3	4	5
6	7	8	9	10
11	12	13	14	15
16	17	18	19	20
21	22	23	24	25

possible bottom right cell

$(N-i) \times (N-j)$
 $= (5-2) \times (5-3)$

$N=5$

since in case of subarray, $(i+1) \times (N-i)$ # of subarrays

submatrices containing $a[i][j] = (i+1) \times (j+1) \times (N-i) \times (N-j)$

how many submatrices element 'i, j' is present = possible TL cell x BR

Quiz

How many submatrices while element (1,2) is present in $N \times M$ matrices, 4×5

$$\begin{aligned} & (i+1) \times (j+1) \times (N-i) \times (M-j) \\ & (1+1) \times (2+1) \times (4-1) \times (5-2) \\ & = 54 \end{aligned}$$

Q2

Find the sum of all submatrices sum.

$$ans = 0$$

```
for (i=0 to N-1) {
  for (j=0 to M-1) {
    ans += (i+1) * (j+1) * (N-i) * (M-j) * a[i][j]
```

}

$$TC: O(N \times M)$$

$$SC: O(1)$$

Assignment

Q2

Search in a row wise and column wise sorted matrix. Given a matrix of integers A of size $N \times M$ and an integer B. In the given matrix every row and column is sorted in non-decreasing order. Find and return the position of B in the matrix in the given form:

- (i) If $A[i][j] = B$ then return $(i * 1009 + j)$
- (ii) If B is not present return -1.

Note 1: Rows are numbered from top to bottom and columns are numbered from left to right

Note 2: If there are multiple B in A then return the smallest value of $i * 1009 + j$ such that $A[i][j] = B$.

Note 3: Expected time complexity is linear

Note 4: Use 1-based indexing

I/p Format: The first argument given is the integer matrix A. The 2nd argument given is the integer B.

O/p Format: Return the position of B and if it is not present in A return -1.

Problem Statement – 1 - Spiral Order Matrix II

Problem Description

Given an integer **A**, generate a square matrix filled with elements from 1 to A^2 in spiral order and return the generated square matrix.

Problem Constraints

$1 \leq A \leq 1000$

Input Format

First and only argument is integer A

Output Format

Return a 2-D matrix which consists of the elements added in spiral order.

Example Input

Input 1: 1

Input 2: 2

Input 3: 5

Example Output

Output 1: `[[1]]`

Output 2:

`[[1, 2],`

`[4, 3]]`

Output 3:

`[[1, 2, 3, 4, 5],`

`[16, 17, 18, 19, 6],`

`[15, 24, 25, 20, 7],`

`[14, 23, 22, 21, 8],`

`[13, 12, 11, 10, 9]]`

Example Explanation

Explanation 1: Only 1 is to be arranged.

Explanation 2:

1 --> 2

|

|

4<--- 3

Explanation 3:

1	2	3	4	5
16	17	18	19	6
15	24	25	20	7
14	23	22	21	8
13	12	11	10	9



Actual Code

```
public class Solution {
    public int[][] generateMatrix(int A) {
        int B = A*A;
        int [][]mat = new int[A][A];
        int r = 0, c = 0;
        int i =0, j = 0;
        int num=1;
        while(A>1){
            i =r;
            j = c;
            for(int k = 0; k< A-1;k++){
                mat[i][j] = num ++;
                j++;
            }
            for(int k =0; k<A-1; k++){
                mat[i][j] = num ++;
                i++;
            }
            for(int k =0; k<A-1; k++){
                mat[i][j] = num++;
                j--;
            }
            for(int k =0; k<A-1; k++){
                mat[i][j] = num++;
                i--;
            }
            r++;
            c++;
            A-=2;
        }
        if(A==1){
            mat[r][c] = num++;
        }
        return mat;
    }
}
```

Problem Statement – 2 - Search in a row wise and column wise sorted matrix

Problem Description

Given a matrix of integers **A** of size **N x M** and an integer **B**.

In the given matrix every row and column is sorted in non-decreasing order. Find and return the position of **B** in the matrix in the given form:

- If $A[i][j] = B$ then return $(i * 1009 + j)$
- If **B** is not present return -1.

Note 1: Rows are numbered from top to bottom and columns are numbered from left to right.

Note 2: If there are multiple **B** in **A** then return the smallest value of $i*1009 + j$ such that $A[i][j]=B$.

Note 3: Expected time complexity is linear

Note 4: Use 1-based indexing

Problem Constraints

$1 \leq N, M \leq 1000$

$-100000 \leq A[i] \leq 100000$

$-100000 \leq B \leq 100000$

Input Format

The first argument given is the integer matrix **A**.

The second argument given is the integer **B**.

Output Format

Return the position of **B** and if it is not present in **A** return -1 instead.

Example Input

Input 1:-

$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$

$B = 2$

Input 2:-

$A = \begin{bmatrix} 1 & 2 \\ 3 & 3 \end{bmatrix}$

$B = 3$

Example Output

Output 1:- 1011

Output 2:- 2019

Example Explanation

Expanation 1:-

$A[1][2] = 2$

$1 * 1009 + 2 = 1011$

Explanation 2:-

$A[2][1] = 3$

$2 * 1009 + 1 = 2019$

$A[2][2] = 3$

$2 * 1009 + 2 = 2020$

The minimum value is 2019

Actual Code

```
public class Solution {
    public int solve(int[][] A, int B) {
        int n = A.length;
        int m = A[0].length;
        if (n==0) return -1;
        int i =0, j=m-1;
        int val = 0, minval = Integer.MAX_VALUE;
        while(i<n && j>=0){
            if(A[i][j] == B){
                val = (i+1)*1009 + (j+1);
                minval = Math.min(minval, val);
                j--;
            }
            else if(A[i][j] > B) j--;
            else i++;
        }
        return (minval != Integer.MAX_VALUE) ? minval: -1;
    }
}
```

Problem Statement – 3 - Sum of all Submatrices

Problem Description

Given a 2D Matrix **A** of dimensions **N*N**, we need to return the sum of all possible submatrices.

Problem Constraints

$1 \leq N \leq 30$

$0 \leq A[i][j] \leq 10$

Input Format

Single argument representing a 2-D array A of size N x N.

Output Format

Return an integer denoting the sum of all possible submatrices in the given matrix.

Example Input

Input 1:

A = [[1, 1]
 [1, 1]]

Input 2:

A = [[1, 2]
 [3, 4]]

Example Output

Output 1: 16

Output 2: 40

Example Explanation

Example 1:

Number of submatrices with 1 elements = 4, so sum of all such submatrices = $4 * 1 = 4$

Number of submatrices with 2 elements = 4, so sum of all such submatrices = $4 * 2 = 8$

Number of submatrices with 3 elements = 0

Number of submatrices with 4 elements = 1, so sum of such submatrix = 4

Total Sum = $4+8+4 = 16$

Example 2:

The submatrices are [1], [2], [3], [4], [1, 2], [3, 4], [1, 3], [2, 4] and [[1, 2], [3, 4]].

Total sum = 40

Actual Code

```
public class Solution {
    public int solve(int[][] A) {
        int n = A.length;
        int m = A[0].length;
        int sum = 0;
        for(int i = 0; i < n; i++){
            for(int j = 0; j < m; j++){
                sum += (i+1)*(j+1)*(n-i)*(m-j)*A[i][j];
            }
        }
        return sum;
    }
}
```

Problem Statement – 4 - Minimum Swaps

Problem Description

Given an array of integers **A** and an integer **B**, find and return the minimum number of swaps required to bring all the numbers less than or equal to **B** together.

Note: It is possible to swap any two elements, not necessarily consecutive.

Problem Constraints

$1 \leq \text{length of the array} \leq 100000$

$-10^9 \leq A[i], B \leq 10^9$

Input Format

The first argument given is the integer array A.

The second argument given is the integer B.

Output Format

Return the minimum number of swaps.

Example Input

Input 1:

A = [1, 12, 10, 3, 14, 10, 5]

B = 8

Input 2:

A = [5, 17, 100, 11]

B = 20

Example Output

Output 1:

2

Output 2:

1

Example Explanation

Explanation 1:

A = [1, 12, 10, 3, 14, 10, 5]

After swapping 12 and 3, A => [1, 3, 10, 12, 14, 10, 5].

After swapping the first occurrence of 10 and 5, A => [1, 3, 5, 12, 14, 10, 10].

Now, all elements less than or equal to 8 are together.

Explanation 2:

A = [5, 17, 100, 11]

After swapping 100 and 11, A => [5, 17, 11, 100].

Now, all elements less than or equal to 20 are together.

Actual Code

```
public class Solution {  
    public int solve(int[] A, int B) {  
        int fav = 0, nonfav = 0, res = Integer.MAX_VALUE;  
        for(int i = 0; i < A.length; i++){  
            if(A[i] <= B) fav++;  
        }  
    }  
}
```

```

    }
    for(int i = 0; i<fav; i++){
        if(A[i] > B) nonfav++;
    }

    int i=0, j = fav-1;
    while (j < A.length){
        res = Math.min(res, nonfav);
        j++;
        if(j<A.length && A[j] > B) nonfav++;
        if(i<A.length && A[i] > B) nonfav--;
        i++;
    }
    return (res == Integer.MAX_VALUE) ? 0: res;
}
}

```

Problem Statement – 5- Row with maximum number of ones

Problem Description

Given a binary sorted matrix **A** of size **N x N**. Find the row with the **maximum** number of **1**.

NOTE:

- If two rows have the maximum number of 1 then return the row which has a **lower index**.
- Rows are numbered from top to bottom and columns are numbered from left to right.
- Assume **0-based** indexing.
- Assume each row to be sorted by values.
- Expected time complexity is $O(\text{rows} + \text{columns})$.

Problem Constraints

$1 \leq N \leq 1000$

$0 \leq A[i] \leq 1$

Input Format

The only argument given is the integer matrix A.

Output Format

Return the row with the maximum number of 1.

Example Input

Input 1:

A = [[0, 1, 1]

[0, 0, 1]
[0, 1, 1]]

Input 2:

A = [[0, 0, 0, 0]
[0, 0, 0, 1]
[0, 0, 1, 1]
[0, 1, 1, 1]]

Example Output

Output 1:

0

Output 2:

3

Example Explanation

Explanation 1:

Row 0 has maximum number of 1s.

Explanation 2:

Row 3 has maximum number of 1s.

Actual Code

```
public class Solution {  
    public int solve(int[][] A) {  
        int index = -1, res = 0;  
        int i = 0, j = A.length - 1;  
        while (i < A.length && j >= 0) {  
            if (A[i][j] == 1) {  
                index = i;  
                //count++;  
                res = Math.max(res, index);  
                j--;  
            }  
            else if (A[i][j] == 0) i++;  
        }  
        return res;  
    }  
}
```